# 04 — dmem.v Explained: The Data Memory

Oct 6, 2026 · @shiv

## 1. Overview

`dmem.v` is 256 bytes of data memory. ST writes a byte at the clock edge; LD reads a byte immediately. It is used only in Stage 4 (MEM).

| Item | Value |
|---|---|
| File | design_sources/dmem.v (48 lines) |
| Module name | dmem |
| Pipeline stage | Stage 4 – Memory access (MEM) |
| Type of logic | Clocked write, combinational read |
| Size | 256 × 8 bits |
| Calls (instantiates) | Nothing – leaf module |
| Called by | risc8_5stage_core.v, instance u_dmem, lines 244–250 |
| Also read by | tb_risc8_5stage.v (dut.u_dmem.mem[8'h20]) to check the store |

### 1.1 Where it sits

```
tb_risc8_5stage.v
└── risc8_5stage_core.v
    ├── imem.v
    ├── control_unit.v
    ├── regfile.v
    ├── hazard_unit.v
    ├── alu.v
    └── dmem.v           <== THIS FILE
```

## 1.2 Ports

| Port | Direction | Width | Meaning | Connected to (in risc8_5stage_core.v) |
|---|---|---|---|---|
| clk | input | 1 | Clock | clk |
| we | input | 1 | Write enable (store) | mem_we = exmem_mem_we && !halted |
| addr | input | 8 | Byte address | exmem_addr (value of rs1) |
| wdata | input | 8 | Data to store | exmem_store_data (value of rd) |
| rdata | output | 8 | Data read | mem_rdata → mem_result → MEM/WB register |

## 1.3 Dependencies

**Modules, functions or tasks called inside this file:** none. It is a leaf module.

| Used by (file) | How | Instance | Lines |
|---|---|---|---|
| risc8_5stage_core.v | Module instance | u_dmem | 244–250 |
| tb_risc8_5stage.v | Reads array mem to check the stored value | dut.u_dmem | 118–120 |

## 1.4 Verilog constructs used

| Construct | Meaning in plain words |
|---|---|
| `reg [7:0] mem [0:255]` | Array of 256 bytes |
| `initial` | Sets the starting contents |
| `always @(posedge clk)` | Clocked write |
| `assign` | Combinational read |

# 2. Code walkthrough, part by part

## 2.1 File header (lines 1–23)

The header explains that LD reads and ST writes this memory, that reading is asynchronous and writing is clocked, and which files use it. Lines 20–22 describe how FPGAs build it: as LUT-RAM (distributed RAM) on Xilinx devices.

## 2.2 Module and ports (lines 25–33)

```
`timescale 1ns/1ps

module dmem (
    input  wire       clk,
    input  wire       we,       // 1 = store this cycle
    input  wire [7:0] addr,     // byte address
    input  wire [7:0] wdata,    // data to store
    output wire [7:0] rdata     // data loaded
);
```

The ports are `clk` for writing, `we` (1 only for a store), an 8-bit address, the data to store and the data read.

## 2.3 The memory array and its starting contents (lines 35–39)

```
    reg [7:0] mem [0:255];
    integer i;

    // Clear memory at start of simulation / FPGA power-up.
    initial for (i = 0; i < 256; i = i + 1) mem[i] = 8'h00;
```

The memory is an array of 256 bytes. The `initial` line clears every byte to 0 at the start of simulation; on an FPGA it sets the power-up contents.

## 2.4 Writing, for ST (lines 41–43)

```
    // WRITE on the rising edge when we = 1.
    always @(posedge clk)
        if (we) mem[addr] <= wdata;
```

At each rising clock edge, if `we` = 1, the byte at `addr` becomes `wdata`.

**Link to another file:** in risc8_5stage_core.v, `we` is `exmem_mem_we && !halted`. It is 1 only when a **valid ST** instruction is in the MEM stage, so bubbles and flushed instructions can never write memory.

### 2.5 Reading, for LD (lines 45–48)

```
    // READ: combinational.
    assign rdata = mem[addr];

endmodule
```

The byte at `addr` comes out immediately. In the core it passes through a multiplexer (`mem_result`) into the MEM/WB register and is written to rd in WB.

## 3. LD and ST through the memory stage

| Instruction | addr comes from | wdata comes from | we | What happens |
|---|---|---|---|---|
| ST R2, [R4] | exmem_addr = R4 (0x20) | exmem_store_data = R2 (0x0F) | 1 | memory[0x20] becomes 0x0F at the clock edge |
| LD R5, [R4] | exmem_addr = R4 (0x20) | not used | 0 | rdata = memory[0x20] = 0x0F, written to R5 in WB |
| Any other instruction | ALU result passes by | not used | 0 | Memory unchanged |

**Why loads cause a stall:** a load's data is ready only at the end of MEM. An instruction that needs it at the start of EX in the next cycle would be too early, so the hazard unit inserts one stall cycle (see Document 06).

## 4. Common mistakes

- Letting a flushed or bubble instruction write memory. The core prevents this by allowing writes only for valid instructions.
- Expecting a loaded value in the very next instruction without a stall.

## 5. Viva questions

**Q1. Which instructions use the data memory?** Only LD (read) and ST (write).

**Q2. Why separate instruction and data memories?** This is a Harvard architecture: IF and MEM can use their own memories in the same cycle, so there is no structural hazard.

**Q3. How is the address formed?** It is the value of register rs1 (register-indirect addressing).

**Q4. What hardware does Vivado use?** Distributed LUT-RAM, which allows an asynchronous read.